SmartCart

SmartCart is an Amazon-inspired e-commerce web platform engineered with a **microservices backend** and a modern **Next.js 15 App Router frontend**. The backend utilizes Node.js, Express, MongoDB (Atlas/Local), and MySQL, decoupled through an intelligent API Gateway supporting both REST and GraphQL.

Group ID: 03 (TeamID 03)

Google Drive Link:- https://drive.google.com/drive/folders/1q9IY8r2_aUCeogiUsT0tpODwt3hF6PHL?usp=sharing
Github Repo Link:- https://github.com/Ayush17314/SmartCart

Name	Roll no.
Ayush Kumar Dubey	20251651030
Abhinav Jain	20251651003
Anuj Gupta	20251651025
Bharath Kumar MP	20251651031
Kaushik Nanda Upadhaya	20251651050

Architecture Overview

```
SmartCart/
├── Frontend/
│   └── smartcart-web/            # Next.js 15 (App Router), React 19, Tailwind CSS v4, Zustand, TanStac
├── Backend/
│   ├── api-gateway/              # Entrypoint (Port 4000) - Routing, Rate Limiting, GraphQL, OpenAPI, 
│   ├── auth-service/             # Identity (Port 4001) - Registration, Login, RBAC, Sessions, Verifica
│   ├── product-service/          # Catalog (Port 4002) - Products, Search, Categories, Inventory, Revie
│   ├── cart-service/             # Shopping Cart (Port 4003) - Cart Items, Quantities, Pricing
│   ├── wishlist-service/         # Saved Items (Port 4004) - Customer Wishlists
│   ├── order-service/            # Orders (Port 4005) - Checkout, Stock Reservations, Returns, Coupons
│   ├── payment-service/          # Payments (Port 4006) - Razorpay Orders/Checkout, Webhooks, Refunds,
│   ├── notification-service/     # Notifications (Port 4007) - Order/Payment/Security alerts via Resend
│   ├── docker-compose.yml        # Multi-container orchestration (Node, MongoDB, MySQL)
│   └── SmartCart.postman_collection.json # API endpoints collection
├── SmartCart-Backend-Checklist-Guide.pdf # Detailed 28-item implementation & architecture guide
└── README.md
```

Web Services Assignments: Implementation Checklist

Below is the verified implementation status organized assignment-wise across the SmartCart system:

Assignment 1: HTTP by Hand & Project Setup

- **Git Repository & Clean Setup:** Implemented (Git repository initialized with structured directories, .gitignore, and incremental commit history)

Assignment 2: Design Project — Services, Contracts & Schema

- **Service Boundaries & Isolation:** Implemented (8 independent microservices: Auth, Product, Cart, Wishlist, Order, Payment, Notification, API Gateway with strict domain separation)
- **Service Operations & Orchestration (`placeOrder`):** Implemented (Order checkout orchestration calling Payment and Cart services with internal coordination hidden behind the API boundary)
- **Database Schema Isolation:** Implemented (Zero shared database collections across services; each service owns its Mongoose schemas, and API Gateway owns its MySQL idempotency/audit store)
- **Contract-Level Communication:** Implemented (All cross-service communication occurs exclusively via HTTP/REST and API Gateway routes, never direct DB access)

Assignment 3: Integrate an External SOAP Partner

- **External Payment Partner Integration:** Partially Implemented (Payment gateway integration exists with order creation, signature verification, and webhooks in payment-service, but implemented using REST/JSON rather than SOAP/XML)
- **Partner WSDL Contract:** Not Implemented (No WSDL contract defining types, messages, portType, and SOAP HTTP binding exists)
- **SOAP Envelopes & Fault Mapping:** Not Implemented (Service does not consume/produce raw SOAP XML envelopes or map SOAP 1.1/1.2 faults)

Assignment 4: Rebuilding a Project Service in REST (Orders Service)

- **Plural Resource Modeling:** Implemented (Plural, resource-oriented endpoints: `/api/orders`, `/api/orders/:id`, `/api/orders/:id/status`, `/api/orders/quote`)
- **OpenAPI Contract Specification:** Implemented (OpenAPI spec defined in `api-gateway/src/openapi.js` and served interactively via Swagger UI at `/docs` and `/openapi.json`)
- **REST Service Implementation:** Implemented (Implemented in Node.js/Express with input validation, distinct HTTP codes, and domain model mapping)
- **Idempotency-Key on Order Creation:** Implemented (API Gateway enforces `Idempotency-Key`, stores initial responses in MySQL/memory, replays identical retries, and returns 409 on payload mismatches)
- **Automated Service Tests:** Implemented (Automated test coverage using Jest/Supertest covering API Gateway, orders, promotions, payments, and notifications)
- **Outbound Network Resilience to Payments:** Implemented (Outbound calls feature timeout protection via `AbortSignal.timeout`, retry with exponential backoff, and circuit breaker in `resilience.js`)

Assignment 5: HTTP Methods & Headers

- **HTTP Method Semantics & Filtering:** Implemented (Standard GET, POST, PATCH, and DELETE routing with query filters and pagination)
- **OPTIONS Method & Allow Header:** Implemented (Explicit `OPTIONS` route handlers returning `204 No Content` and explicit `Allow` header implemented across API Gateway, Orders Service, and Product Service; CORS configured with `preflightContinue`)
- **Bearer Authentication & Security Headers:** Implemented (JWT Bearer tokens required on protected routes, CORS allowlist enforced, and Helmet security headers enabled)
- **Content Negotiation (Accept Header):** Implemented (Accept header negotiation across API Gateway, Product Service, and Orders Service supporting `application/json` and `application/xml` with `Vary: Accept`; unsupported Accept headers rejected with `406 Not Acceptable`)
- **ETag Caching & Conditional 304:** Implemented (Dynamic ETags generated on GET requests for single items across Products and Orders; `If-None-Match` returns `304 Not Modified`)
- **Precondition Checking (If-Match / 412):** Implemented (Optimistic concurrency with `If-Match` header validation on order status updates, order cancellations, and product updates; returns `412 Precondition Failed` if stale)
- **Rate Limiting & 429 Responses:** Implemented (Standard rate-limiting middleware returns `429 Too Many Requests` along with standard `RateLimit-*` headers, `Retry-After` header in seconds, and Problem Details envelope)

Assignment 6: Honest Responses: Status Codes, Errors & Validation

- **Honest HTTP Status Codes:** Implemented (Endpoints return accurate HTTP codes 200, 201, 304, 400, 401, 403, 404, 406, 409, 412, 422, 429, 500, 503, and 504 instead of 200 with error payloads)
- **Location Header on 201 Created:** Implemented (`Location` header pointing to canonical resource URI set on `POST /api/orders` and `POST /api/products`, and forwarded by API Gateway)
- **RFC 7807/9457 Problem Details (`application/problem+json`):** Implemented (Unified `problem()` helper across services producing standard `type`, `title`, `status`, `detail`, `instance`, and `requestId`, with `Content-Type: application/problem+json`)
- **Error Masking & Types:** Implemented (Internal errors safely masked with request IDs; stable machine-readable URI type catalogue enforced for domain errors: `empty-cart`, `item-unavailable`, `payment-declined`, `order-not-found`, `illegal-transition`, `precondition-failed`, etc.)
- **Multi-Field Validation Array:** Implemented (AJV validation captures all errors simultaneously returning structured `errors[]` array with `field`, `message`, and `keyword`; empty checkout returns `422 Unprocessable Entity` with `errors[]`)
- **JSON and XML Representation Negotiation:** Implemented (Both Product Service and Orders Service support JSON and XML representation negotiation via `Accept` headers with `Vary: Accept`, returning `406 Not Acceptable` if unsupported)

Services & Ports Reference

Service	Port	Database	Primary Responsibility
API Gateway	`4000`	MySQL (`smartcart`)	Routing (`/v1/api` & `/api`), Rate Limiting, OpenAPI/Swagger, GraphQL, Idempotency, Request Audits
Auth Service	`4001`	MongoDB (`smartcart_auth`)	User signup, login, Google OAuth, Email verification, RBAC permissions, Staff invites
Product Service	`4002`	MongoDB (`smartcart_products`)	Product catalog, Category filters, Search, Inventory management, Customer reviews
Cart Service	`4003`	MongoDB (`smartcart_cart`)	Customer shopping carts, Item quantities, Cart totals
Wishlist Service	`4004`	MongoDB (`smartcart_wishlist`)	Customer saved product wishlists
Order Service	`4005`	MongoDB (`smartcart_orders`)	Order placement, Stock reservations/rollbacks, Coupon discounts, Shipment tracking, Returns
Payment Service	`4006`	MongoDB (`smartcart_payments`)	Razorpay order creation, Signature verification, Webhook processing, Refund management, COD
Notification Service	`4007`	MongoDB (`smartcart_notifications`)	Transactional email delivery (Resend / Mock), Delivery logs, Notification preferences
Frontend Web App	`3000`	—	Next.js 15 App Router storefront, Customer portal, Staff & Admin management suite

Roles & Access Control (RBAC)

SmartCart enforces a strict Role-Based Access Control (RBAC) hierarchy. Backend microservices validate granular permission claims issued by the Auth Service, and dedicated portal gates prevent privilege crossover. No sensitive user credentials, account identifiers, or passwords are hardcoded in the codebase.

Role	Access Portal	Assigned Permissions	Responsibilities & Capabilities
`customer`	Storefront (`/login` , `/account`)	`products.read` , `orders.read_own` , `refunds.request`	• Default account assigned upon registration. • Browse, search, filter, and view catalog products and customer reviews. • Manage cart and wishlist items, apply server-validated coupons. • Place orders via Razorpay (UPI/Card) or Cash on Delivery (COD). • View real-time shipment tracking for their own orders. • Request order returns within 30 days of delivery and submit product reviews.
`support`	Staff Portal (`/admin/login`)	`products.read` , `orders.read_all` , `customers.read` , `returns.review` , `refunds.request` , `notifications.manage`	• Customer service and support desk operations. • Access order records across all customers to diagnose order inquiries. • Review customer return requests (approve or reject with reasons). • Initiate refund requests for approved returns. • Inspect customer profiles and audit notification delivery logs.
`inventory_staff`	Staff Portal (`/admin/login`)	`products.read` , `products.write` , `inventory.read` , `inventory.write` , `orders.read_all`	• Warehouse and catalog operations. • Create, update, and manage product listings, pricing, and category assignments. • Monitor inventory levels and low-stock threshold alerts. • Record stock adjustments, restocks, and inspected returned inventory. • View incoming and active customer orders to prepare shipments.

Role	Access Portal	Assigned Permissions	Responsibilities & Capabilities
`admin`	Admin Suite (`/admin`)	All staff permissions plus `orders.manage` , `customers.manage` , `promotions.manage` , `reviews.moderate` , `staff.read` , `analytics.read`	• General e-commerce store operations. • Advance order lifecycles (`CONFIRMED` -> `SHIPPED` -> `DELIVERED`). • Attach carrier tracking details to dispatches. • Create and manage coupon promotions, discount types, and usage limits. • Moderate customer reviews (publish/hide). • View customer accounts and export paginated sales reports (CSV).
`super_admin`	Super Admin Suite (`/super-admin`)	Complete system permissions (`refunds.approve` , `staff.manage` , `settings.manage` , `audit.read` , etc.)	• Top-tier administrative governance protected by multi-step login challenges. • Provision, invite, activate, and manage staff accounts and roles (`/admin/staff`). • Authorize and execute financial payment refunds through Razorpay. • Manage store identity, tax rates, shipping rules, and return policy settings. • Review immutable security, transaction, and gateway audit logs.

Order & Payment Lifecycle

1. Online Payments (UPI & Card via Razorpay)

1. **Order Creation:** Client submits `POST /api/orders` . Order Service validates prices, reserves inventory in Product Service, calculates taxes/shipping, and calls Payment Service (`POST /api/payments/create`).
2. **Payment Intent:** Payment Service creates a Razorpay Order and returns checkout options (`orderId` , `amount` , `keyId`).
3. **Verification:** The frontend opens the Razorpay modal. Upon completion, it sends `POST /api/payments/verify/:paymentId` . Payment Service validates the cryptographic HMAC-SHA256 signature and queries Razorpay's API to confirm `status === 'captured'` .

4. **Order Confirmation:** Payment Service notifies Order Service via `PATCH /internal/orders/:id/payment`, setting `paymentStatus: "SUCCESS"` and committing reserved inventory. Webhooks (`POST /api/payments/webhook`) serve as a resilient fallback if the client window closes.

2. Cash on Delivery (COD)

1. Order is created with `paymentMethod: "COD"`. Payment Service marks gateway as `OFFLINE` and payment status as `PENDING`.
2. Admin confirms and ships the order (`CONFIRMED` -> `SHIPPED`), which automatically commits stock.

Getting Started

Prerequisites

- Node.js (v20+ recommended)
- pnpm (v9+)
- Docker Desktop (for Docker Compose) or local MongoDB & MySQL instances

Option A: Running with Docker Compose (Recommended)

1. **Clone and setup environment variables:**
   `bash cp Backend/.env.example Backend/.env`
   (Update `MONGODB_URI`, `JWT_SECRET`, and Razorpay test credentials if testing payments).

2. **Start the backend stack:**
   `bash cd Backend docker compose up --build`
   * API Gateway will be live at `http://localhost:4000`
   * Swagger Documentation will be accessible at `http://localhost:4000/docs`

3. **Start the Frontend:**
   `bash cd ../Frontend/smartcart-web pnpm install pnpm dev`
   * Storefront & Admin interface will be live at `http://localhost:3000`

Option B: Running Locally Without Docker

1. **Start MongoDB and MySQL locally.**
2. **Install and run API Gateway & Services:**
   In each service folder (`Backend/api-gateway`, `Backend/auth-service`, `Backend/product-service`, etc.):
   `bash npm install npm run dev`
3. **Run the Next.js Frontend:**
   `bash cd Frontend/smartcart-web pnpm install pnpm dev`

Testing & Verification

Running Gateway & Integration Tests

```
cd Backend/api-gateway
npm test
```

The test suite covers:
* Pre-proxy AJV payload validation
* Versioned `/v1/api` routing & ETag conditional 304 caching
* Idempotent POST replay and conflict detection
* Rate-limit header verification
* Circuit breaker failure tripping
* Accept header XML content negotiation
* Uniform error envelopes with correlation request IDs

Postman Collection

Import `Backend/SmartCart.postman_collection.json` into Postman to inspect and test all microservice REST routes.